

Лабораторная работа №7
по дисциплине «Управление качеством программного обеспечения»
«Метрики Лоренца и Кидда»
Вариант 2

Задача 6. Определить понятие «Абонент телефонной сети». Состояние объекта определяется полями:

- номер телефона (семизначное целое число);
- фамилия (строка до 20 символов);
- номер паспорта (длинное целое число).

Базируясь на указанном понятии, определить понятие «Задолженность», дополнительно определив понятие «Абонент телефонной сети» полем «Долг по оплате» (вещественное число). По таблице абонентов вывести номера телефонов и фамилии абонентов, у которых долг по оплате превышает заданный предел.

```
import java.util.ArrayList;
import java.util.List;
class PhoneSubscriber {
    protected int phoneNumber;
    protected String surname;
    protected long passportNumber;
    public PhoneSubscriber(int phoneNumber, String surname, long passportNumber) {
        if (phoneNumber < 1000000 || phoneNumber > 9999999) {
            throw new IllegalArgumentException("Номер телефона должен быть семизначным");
        }
        if (surname.length() > 20) {
            throw new IllegalArgumentException("Фамилия не должна превышать 20 символов");
        }
        this.phoneNumber = phoneNumber;
        this.surname = surname;
        this.passportNumber = passportNumber;
    }
    public int getPhoneNumber() {
        return phoneNumber;
    }
    public String getSurname() {
        return surname;
    }
    @Override
    public String toString() {
        return String.format("Телефон: %d, Фамилия: %s, Паспорт: %d", phoneNumber, surname,
passportNumber);
    }
}
class Debt extends PhoneSubscriber {
    private final double debtAmount;
    public Debt(int phoneNumber, String surname, long passportNumber, double debtAmount)
    {
        super(phoneNumber, surname, passportNumber);
    }
}
```

```

    this.debtAmount = debtAmount;
}
public double getDebtAmount() {
    return debtAmount;
}
@Override
public String toString() {
    return String.format("Телефон: %d, Фамилия: %s, Паспорт: %d, Долг: %.2f",
phoneNumber, surname, passportNumber, debtAmount);
}
}
class SubscriberManager {
    private final List<Debt> subscribers;
    public SubscriberManager() {
        this.subscribers = new ArrayList<>();
    }
    public void addSubscriber(Debt subscriber) {
        subscribers.add(subscriber);
    }
    public void printSubscribersWithDebtAbove(double debtLimit) {
        System.out.println("Абоненты с долгом выше " + debtLimit + ":");
        boolean found = false;
        for (Debt subscriber : subscribers) {
            if (subscriber.getDebtAmount() > debtLimit) {
                System.out.printf("Телефон: %d, Фамилия: %s, Долг: %.2f\n",
                    subscriber.getPhoneNumber(),
                    subscriber.getSurname(),
                    subscriber.getDebtAmount());
                found = true;
            }
        }
        if (!found) {
            System.out.println("Абонентов с долгом выше " + debtLimit + " не найдено.");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        SubscriberManager manager = new SubscriberManager();
        manager.addSubscriber(new Debt(1234567, "Иванов", 1234567890L, 1500.50));
        manager.addSubscriber(new Debt(2345678, "Петров", 2345678901L, 800.25));
        manager.addSubscriber(new Debt(3456789, "Сидоров", 3456789012L, 2500.75));
        manager.addSubscriber(new Debt(4567890, "Кузнецов", 4567890123L, 500.00));
        manager.addSubscriber(new Debt(5678901, "Смирнов", 5678901234L, 1800.30));
        manager.printSubscribersWithDebtAbove(1000.0);
    }
}

```

Размер класса (CS) = $C_{\Sigma} + S_{\Sigma}$ = количество инкапсулированных классом методов + количество инкапсулированных классом свойств = $0 + 1 = 1 \leq 20$.

Количество операций, переопределяемых подклассом NOO = $1 \leq 3$

Количество операций, добавленных подклассом $NOA = 1 \leq 4$

Индекс специализации $SI = NOO \cdot u / M_{\text{общ}} = 1 \cdot \text{номер уровня в иерархии, на котором находится подкласс} / \text{общее количество методов класса} = 1 \cdot 2 / 3 = \underline{2 / 3} \geq 0.15$

Вероятность нарушения абстракции базового класса отдельными объектами высока.

Средний размер операции $AOS = \text{количество сообщений, порождаемых операциями} = \text{super()} + \text{String.format()} = 2 \leq 9$

Сложность операции ОС:

Действие	Вес	Debt	getDebtAmount	toString
Определение (описание) переменной-параметра	0,3	0	0	0
Определение (описание) временной переменной	0,5	0	1·0,5	0
Присваивание значения	0,5	1·0,5	0	0
Вложенное выражение	0,5	0	0	0
Сообщение без параметров	1	0	0	0
Арифметическая операция	2	0	0	0
Сообщение с параметрами	3	1·3	0	0
Вызов стандартной функции интерфейса (API)	5	0	0	1·5
Вызов пользовательской функции (простой вызов)	7	0	0	0
Итого		$3,5 \leq 65$	$0,5 \leq 65$	$5 \leq 65$

Среднее количество параметров на операцию ANP = количество параметров / количество операций = $4 / 4 = \underline{1} > 0,7$

Количество описаний сценариев NSS = количество методов класса = $3 > 1$

Количество ключевых классов программы НКС = $1 > 0,2$, поскольку все классы программы непосредственно связаны с проблемной областью.

Количество подсистем NSUB = подсистема абонента + подсистема должника + подсистема таблицы абонента = $3 \geq 3$